AI Assignment & Deadline Manager

Technical Report | Ollama + Python | Group of Two

Student 1: Lasitha Prabashwara / CCLARKE562 Student 2: Chathumini Hansika / CCLARKE543

Submission date: 24 October 2026

Abstract

This project implements a locally running AI Assignment and Deadline Manager for students who must track coursework, estimate effort and prepare study schedules. The application uses Python and Ollama language model. Rather than only answering questions, the agent chooses from six callable tools, receives structured results and continues until a request is complete. Assignments are stored in a local JSON file. A greedy earliest-deadline-first planner assigns available study hours to unfinished tasks. The supplied demonstration traces show successful assignment creation, retrieval, workload calculation, study planning, status updates and invalid-date rejection. This report distinguishes observed behavior from limitations and outstanding verification work. It also documents the use of an AI coding assistant.

1. Problem and Scenario

Students often struggle to manage multiple assignments with different deadlines, priorities, and workloads. Manual tracking makes it difficult to organize tasks and plan study time effectively. The proposed AI Assignment & Deadline Manager allows students to add assignments, check deadlines, calculate workloads, and generate study plans using natural-language commands. The system runs locally using Ollama, without paid cloud APIs, focusing on efficient task management, AI tool execution, and data privacy.

2. Objectives and Scope

The main objective of this project is to help students manage their assignments easily using AI. The system allows users to add assignments, check deadlines, update task status, calculate workloads, and create study plans. It can also perform multiple tasks from a single request. The system runs locally using Ollama, saves assignment details, and handles errors. Advanced features such as notifications, calendar integration, and multiple-user access are not included to keep the project simple and focused on AI agent functionality.

3. Architecture and Data Flow

The system uses Python and Ollama with the Qwen2.5 model. Users enter requests, and the AI selects the correct tool to complete them. The results are returned to the AI to generate a response. Tool calls are displayed and logged so we can check how the system works.

4. Tool Design

We developed six main tools:

- **Add Assignment:** Save new assignments with deadlines and priorities.
- **List Assignments:** Display pending or completed assignments.
- **Check Deadlines:** Find upcoming and overdue assignments.
- **Update Status:** Mark assignments as pending or completed.
- **Calculate Workload:** Calculate the total study hours needed.
- **Create Study Plan:** Divide study hours across available days.

5. Agent Loop and Prompt Design

The AI understands the user's request and selects the required tools. It can perform multiple operations from one message. The system keeps conversation history during a session and limits tool execution to 10 rounds to prevent repeated calls.

The system instruction tells the model to use tools for task operations, not invent successful outcomes, request missing essential information and explain tool errors. The controller includes the current date with user messages so relative time expressions have a reference point. Function schemas provide descriptions, parameter types, accepted priority and status values, and required fields. The controller appends the assistant tool-call message and then the corresponding tool result message to preserve the context needed for follow-up decisions. The ten-round bound prevents unlimited repeated calls. Conversation history persists while the command-line program remains open, allowing follow-up questions to refer to earlier actions, but it is not saved across program restarts. The model's choice of tool arguments is probabilistic rather than guaranteed. In an initial observed run, a seven-day request triggered check_deadlines with days set to one. A more explicit prompt produced the correct seven-day call. This example illustrates why visible traces and verification of arguments are necessary even when the final natural-language answer appears plausible.

6. Storage, Validation and Error Handling

Assignment details are saved in a local JSON file. The system checks inputs such as dates, priorities and estimated hours before saving them. Invalid inputs return error messages, while tool activities are recorded in a log file.

7. Study-Planning Algorithm

The study planner gives priority to assignments with earlier deadlines. It divides the estimated workload according to the student's available daily study hours and avoids scheduling work after its deadline.

The planner uses a greedy earliest-deadline-first allocation method. It first sorts pending tasks by due date, breaking ties with high, medium and low priority. It creates a fixed number of daily slots beginning with the current date. For each day, it starts with the configured daily study-hour and allocates as much time as possible to the earliest eligible task. The task's remaining estimated hours and the day's remaining capacity are reduced. Allocation stops when the day's capacity is exhausted or no eligible work remains. A task is not scheduled after its deadline. The output includes a list of dates and sessions, plus a mapping of tasks with hours left unscheduled.

8. Testing and Observed Results

We tested adding assignments, listing tasks, checking deadlines, calculating workloads, generating study plans and updating task status. The system successfully completed these operations. We also tested an invalid date, and the system returned an error instead of saving incorrect information.

9. Challenges, Limitations and Improvements

One challenge was getting the AI to select the correct tools and parameters. Sometimes it misunderstood the requested time period. The system also depends on the computer's performance. Currently, it supports one user, stores data locally and does not provide automatic reminders.

The JSON store is suitable for a demonstration but would need a database and transaction handling for multiple users. Conversation history is lost on exit. The planner assumes that estimated hours are accurate and that each day has equal availability; its greedy method is easy to explain but not an optimal personal timetable. A future version could add editable daily availability, overdue warnings, a SQLite database, a graphical interface, persistent conversational state and optional calendar integration. Such integrations should remain separate from the locally running core model and should be added only after more systematic tests.

10. Ethics, AI Assistance and Individual Contributions

Our project focuses on protecting user information and providing reliable results. Assignment details are stored locally in a JSON file instead of being sent to an online database. We use Ollama to run the AI model locally, which helps maintain privacy. The system also validates user inputs and displays tool execution results to make its operations more transparent.

We used ChatGPT as a supporting tool during project development. It helped us understand the AI agent concept, prepare the initial Python code, identify errors and improve our documentation. We used Ollama with the Qwen2.5 model to process user requests and select the appropriate tools. We also tested the main functions to check whether the system worked as expected. AI assistance was useful for development, but we are responsible for reviewing, understanding and explaining our implementation.

We used ChatGPT to help with the initial coding and documentation. We tested the system and reviewed its functionality. Both group members are responsible for understanding the code and explaining the project during the viva.

Group Member 1: Lasitha Prabashwara / CCLARKE562

Group Member 2: Chathumini Hansika / CCLARKE543

11. Conclusion

Our project helps students manage assignments and study time using a locally running AI agent. It can understand natural-language requests, execute different tools and generate useful results. The project demonstrates how an AI agent can perform practical tasks through Python tools. Final submission readiness depends on verifying the repository, unit tests, group contributions, logging and report placeholders rather than on adding more features.